Ex. No: 1 **Longest Unique Substring**

PROGRAM:

```
import java.util.Scanner;
public class UniqueSubstring {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        HashMap<Character, Integer> map = new HashMap<>();
        int left = 0, max = 0;
        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            if (map.containsKey(ch) && map.get(ch) >= left) {
                left = map.get(ch) + 1;
            }
            map.put(ch, right);
            max = Math.max(max, right - left + 1);
        }
        System.out.println(max);
    }
}
```

Ex. No: 2 **Longest Palindromic Substring**

PROGRAM:

```
import java.util.Scanner;
public class LongestPalindromeSubstring {
    static String expand(String s, int l, int r) {
        while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
            l--;
            r++;
        }
        return s.substring(l + 1, r);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String ans = "";
        for (int i = 0; i < s.length(); i++) {
            String odd = expand(s, i, i);
            if (odd.length() > ans.length())
                ans = odd;
            String even = expand(s, i, i + 1);
            if (even.length() > ans.length())
                ans = even;
        }
        System.out.println(ans);
    }
}
```

Ex. No: 3 **Count Palindromic Substrings**

PROGRAM:

```
import java.util.Scanner;
public class CountPalindromeSubstrings {
    static int count(String s, int l, int r) {
        int c = 0;
        while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
            c++;
            l--;
            r++;
        }
        return c;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int ans = 0;
        for (int i = 0; i < s.length(); i++) {
            ans += count(s, i, i);
            ans += count(s, i, i + 1);
        }
        System.out.println(ans);
    }
}
```

Ex. No: 4 Lexicographically Smallest and Largest Substring

PROGRAM:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int k = sc.nextInt();
        String small = s.substring(0, k);
        String large = s.substring(0, k);
        for (int i = 1; i <= s.length() - k; i++) {
            String sub = s.substring(i, i + k);
            if (sub.compareTo(small) < 0)
                small = sub;
            if (sub.compareTo(large) > 0)
                large = sub;
        }

        System.out.println(small);
        System.out.println(large);
    }
}
```

Ex. No: 5 **Longest Common Prefix**

PROGRAM:

```
import java.util.Scanner;
public class LongestCommonPrefix {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();
        String[] arr = new String[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextLine();
        String prefix = arr[0];
        for (int i = 1; i < n; i++) {
            while (!arr[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.length() == 0) {
                    System.out.println("-1");
                    return;
                }
            }
        }
        System.out.println(prefix);
    }
}
```

Ex. No: 6 Smallest Window Containing All Characters

PROGRAM:

```
import java.util.Scanner;
public class SmallestWindowCharacters {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String p = sc.nextLine();
        int[] need = new int[256];
        int[] have = new int[256];
        for (char c : p.toCharArray())
            need[c]++;
        int left = 0;
        int count = 0;
        int minLen = Integer.MAX_VALUE;
        int start = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            have[c]++;
            if (need[c] > 0 && have[c] <= need[c])
                count++;
            while (count == p.length()) {
                if (right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    start = left;
                }
```

```
            char ch = s.charAt(left);
            have[ch]--;
            if (need[ch] > 0 && have[ch] < need[ch])
                count--;
            left++;
        }
    }
    if (minLen == Integer.MAX_VALUE)
        System.out.println("-1");
    else
        System.out.println(s.substring(start, start + minLen));
  }
}
```

Ex. No: 7 Remove Duplicate Characters While Preserving Order

PROGRAM:

```
import java.util.Scanner;
public class DuplicateCharacters {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        HashSet<Character> set = new HashSet<>();
        StringBuilder ans = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (!set.contains(c)) {
                set.add(c);
                ans.append(c);
            }
        }
        System.out.println(ans);
    }
}
```

Ex. No: 8 Find the Most Frequent Substring

PROGRAM:

```
import java.util.Scanner;
public class FrequentSubstring {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int k = sc.nextInt();
        HashMap<String, Integer> map = new HashMap<>();
        for (int i = 0; i <= s.length() - k; i++) {
            String sub = s.substring(i, i + k);
            map.put(sub, map.getOrDefault(sub, 0) + 1);
        }
        String ans = " ";
        int max = 0;
        for (String key : map.keySet()) {
            int f = map.get(key);
            if (f > max || (f == max && key.compareTo(ans) < 0)) {
                max = f;
                ans = key;
            }
        }
        System.out.println(ans);
    }
}
```

Ex. No: 9 **Circular String Rotation Check**

PROGRAM:

```
import java.util.Scanner;
public class RotationCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = sc.nextLine();
        String s2 = sc.nextLine();
        if (s1.length() == s2.length() && (s1 + s1).contains(s2))
            System.out.println("Yes");
        else
            System.out.println("No");
    }
}
```

Ex. No: 10 **Maximum Vowels in Any Substring**

PROGRAM:

```
import java.util.*;
public class MaximumVowels {
   static boolean isVowel(char c) {
      return "aeiouAEIOU".indexOf(c) != -1;
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String s = sc.nextLine();
      int k = sc.nextInt();
      int count = 0;
      for (int i = 0; i < k; i++) {
         if (isVowel(s.charAt(i)))
            count++;
      }
      int max = count;
      for (int i = k; i < s.length(); i++) {
         if (isVowel(s.charAt(i)))
            count++;
         if (isVowel(s.charAt(i - k)))
            count--;
         max = Math.max(max, count);
      }
      System.out.println(max);
   }
```